

Problema 1

1) Diseñar algoritmo

El código se encuentra en el repositorio como:

ArregloOrdenado.java

2) Explicación del caso base

El caso ocurre en “if(i == a.length-1)”, significando que el algoritmo llegó al último elemento del arreglo sin problemas, osea llegó hasta esa posición final sin encontrar elementos fuera de orden, entonces el arreglo está ordenado de menor a mayor, por lo que se imprime “true”.

3) Obtener T(n)

En cada llamada recursiva el algoritmo realiza una comparación en “if(a[i] > a[i+1])” y una llamada recursiva en “ordenado(a, i+1);”, siendo que:

$T(n) = T(n-1) + c$, Aplicamos varias veces:

$T(n) = T(n-2) + 2c$ — $T(n) = T(n-3) + 3c$ — $T(n) = T(n-4) \dots$ osea: $T(n) = T(n-k) + kc$

se sustituye: $n-k=1$

$T(n) = T(1) + (n-1)c$ quedándonos con: $T(n) = O(n)$

4) Complejidad temporal

La complejidad temporal es “O(n)”, porque cuando el arreglo está completamente ordenado, el algoritmo debe recorrer todos los elementos hasta llegar al final.

Problema 2

1) Diseñar algoritmo

El código se encuentra en el repositorio como:

SegundoNumGrande.java

2) Explicar cómo se combinan los resultados

Se comparan los resultados obtenidos de la parte izquierda y la parte derecha del arreglo si el número de la izquierda es más grande, se toma ese, caso contrario, se toma el de la derecha, siendo que así se obtiene el número mayor del arreglo completo, después de encontrar el número mayor, se vuelve a recorrer el arreglo una última vez para buscar el segundo número más grande, evitando tomar nuevamente el valor mayor encontrado.

3) Obtener $T(n)$

$T(n)=2T(n/2)+c$ Identificando: $a=2, b=2, f(n)=c$

Calculamos:

$$n^{\log_2 2}=n$$

Como:

$c=O(1)$ y $O(1)$ crece más lento que n , se aplica el Caso 1:

$$T(n)=\Theta(n)$$

4) Analizar complejidad temporal Big O

La complejidad temporal es $O(n)$, porque el arreglo se divide repetidamente y cada elemento participa una sola vez en las comparaciones principales, siendo que el proceso sea más fácil.

Problema 3

1) Diseñar algoritmo

El código se encuentra en el repositorio como:

ContarInversiones.java

2) Explicar cómo se detectan las inversiones

El arreglo se divide en dos partes (como el problema anterior): izquierda y derecha. Primero se cuentan las inversiones de cada mitad mediante llamadas recursivas y luego se comparan los elementos de ambas partes **if(num[i] > num[j])**, si un elemento de la izquierda es mayor que uno de la derecha, significa que existe una inversión y el contador aumenta en 1.

3) Obtener la recurrencia $T(n)$

El algoritmo divide el arreglo en dos partes ($2T(n/2)$) donde luego compara elementos entre ambas mitades siendo de n^2 , donde los valores $a=2, b=2, f(n)=n^2$.

Ahora calculamos:

$$n^{\log_2 2}a$$

Sustituyendo con los valores nos da $n^{\log_2 2}a$ donde en $\log_2 2=1$ nos queda que $n^1=n$

Ahora comparamos $f(n)=n^2$ con $n^{\log_2 2}a=n$ y observamos que n^2 y crece más rápido que n siendo que :

$f(n)=\Omega(n^{1+\epsilon})$ tomando: $\epsilon=1$, por lo tanto se cumple el **Caso 3 del Teorema Maestro**

$$T(n)=\Theta(f(n)) \text{ reemplazando } T(n)=\Theta(n^2) \text{ quedando finalmente } O(n^2)$$

4) Analizar complejidad temporal

La complejidad temporal es de $O(n^2)$, porque durante la combinación se realizan comparaciones mediante dos ciclos anidados:

```
for(int i=inicio;i<=mitad;i++){  
    for(int j=mitad+1;j<=fin;j++){
```

Siendo que en el peor caso se revisarán muchas combinaciones entre las dos mitades.

5) Explicar por qué el algoritmo es más eficiente que revisar todos los pares posibles

El algoritmo divide el problema en partes más pequeñas antes de realizar las comparaciones, lo que evita revisar los elementos uno por uno desde el inicio, de esta manera, optimiza el proceso de búsqueda y ordenamiento, considero que en arreglos grandes, este enfoque mejora el rendimiento, el código ayuda a reducir la cantidad de operaciones necesarias.